Foundations for Content-Based Image Database Systems

Lionel Brunie[1], Solomon Atnafu[1], and Harald Kosch[2]
[1]Laboratory of Information Systems Engineering (LISI)
National Institute of Applied Sciences of Lyon (INSA de Lyon)
20 Avenue Albert Einstein; 69621 Villeurbanne, France
{lionel.brunie, satnafu}@lisi.insa-lyon.fr
[2]Department of Information Technology, University of Klagenfurt
Universitätsstr. 65-67; 9020 Klagenfurt, Austria
harald@itec.uni-klu.ac.at

Abstract

The relational database systems have been very successful to handle alphanumeric data during the last three decades. The traditional approach to manage image data however has been by the use of metadata like annotations of image data contents. The representation of image data contents using metadata is found to be incomplete and difficult to design and manage. Hence, supplementing this approach with content-based image data retrieval technique is an important task. The combined and effective use of metadata representation and content-based description of image data and the integration of these into the current popular DBMS can enhance the efficiency of image data management. However, in order to integrate content-based retrieval techniques into the traditional DBMS, the fundamental requirements such as an appropriate image data repository model and the necessary formalism of content-based operations on image data need to be developed. In this report, we first demonstrate the need for introducing novel content-based database operators with example queries. Then, we present similarity-based image retrieval techniques and our image data repository model. Consequently, we present our novel similarity-based operators on image tables and study their properties and possibilities of using them in association with relational operators. Furthermore, we will present the method and possibilities of extending our image repository model to support similarity-based operations on salient objects. Finally, based on the properties of these novel operators, we derive algebraic rules that are useful for similarity-based query optimization and will introduce a cost model for an implementation of one of the major similarity-based operators. Thus, our work lays down the foundations for an image database system that integrate both the traditional and the content-based data management techniques.

Keywords: Image Database, Image Data Repository Model, Similarity-based operators, Similarity-based Algebra, Similarity-based Query Optimization.

1 Introduction

The traditional approach for multimedia data retrieval uses manual annotations of the visual features with the help of meta-data. However, describing the content of a multimedia data (image, video, sound, etc.) using this approach often leads to an incomplete representation.

Moreover, for a large collection of digital media data, a meta-data representation is a tiresome and time-consuming work particularly at the database design and creation stage.

During the last decade, content-based retrieval has been experimented and has been used as an alternative or supplementary method of image data retrieval. Thus, the need for an automatic extraction and manipulation of the content of image data is of critical importance for an efficient media data management. To satisfy this need and to facilitate similarity-based search operations, significant research works have been done (see the survey in [19]). As a result of these works, similarity-based search has become a promising approach for image and video data management systems [24, 4, 25, 19, 26]. The problem of extracting the content of an image, and identifying the proper techniques for comparing, whether two images are alike or not, is more of a work in the field of visual recognition. In this paper, we focus on content-based[1] image database management.

Currently, a large number of digital images are being produced daily, for different purposes in the fields of medicine, crime prevention, military, architectural and engineering design, journalism, geographic information system, etc. Hence, the volume of image data being collected and managed is increasing from time to time. Thus, the need for a systematic and efficient management of image data is very critical. For this reason, a number of works are being done to integrate image data in database management systems. What has been practiced most since the last two decades is, to use keyword like annotations to represent the images [22]. However, assignment of keywords is subjective and the choice of words can vary from person to person. Moreover, image contents such as color, texture and shape cannot be precisely described using keywords. To overcome most of these problems, people in the domain of computer vision have been conducting research and experiment to represent images by its low-level features of color, texture, shape, etc. These features are then used for content-based image retrieval. As a result, currently there are a number of research prototypes and commercial systems that support content-based image retrieval [9]. What most of them do in common is that, for a given single query image or a feature vector representation[2] of an image, they search for its most similar images from a set of images or from an image database [19].

This can be associated with a similarity-based selection operation. Other more complex operations such as a many to many image matching like a similarity-based join are not supported. However, we believe the need to have a formal framework for such similarity-based operations on image databases just like the formalism for the relational database operations. Although content-based retrieval technique can effectively use low-level features of image data, we cannot ignore the role of using some keyword like annotations to semantically describe the images for a more efficient image retrieval system.

Thus, an effective content-based image database system must be able to integrate the capabilities of the traditional database systems with the features of content-based image retrieval systems. The trend is then to use content-based matching in addition to keyword matching. We therefore, investigate the way of using these similarity-based operations in conjunction with the relational operators under an Object-Relational (OR) paradigm. Relational DBMS is quite successful in handling alphanumeric data, but not efficient to handle the complex new applications such as that of managing image data. The new applications require more complex data structures, new data types for storing image data, and the ability to define non-standard application specific operations. Hence, the features of an ORDBMS fits most for a content-based image database systems, since it can specify both the structure of complex objects and the operations that can be applied on them. The use of an OR paradigm, enables us to exploit the techniques

[1] *In this paper the terms "content-based" and "similarity-based" are used interchangeably.*

[2] *i.e. a vector representation of an image in terms of its features such as color, texture, shape, etc.*

and capabilities of Object-Oriented concept in managing data. The capability of an OR DBMS to support image data is widely discussed in [23].

In this paper, we first propose an image repository model that can facilitate the use of both content-based and traditional data retrieval techniques. Then, we define the major required similarity-based operators and study their properties. Based on the properties of these operators, we furthermore study similarity-based query optimization strategies. Since similarity-based image database systems are relatively new domains of research, no much work is done on similarity-based query optimization. Most in this domain have focused on the efficiency of content-based retrieval algorithms and on indexing techniques. We however have identified, novel rules of transformation based on the properties of the novel operators. These rules can be utilized for similarity-based query optimization purpose.

The rest of the sections are organized as follows. Section 2 demonstrates some practical problems where one needs to deal with a combination of content-based and relational operators. Section 3 presents the related work. Section 4 briefly summarizes the methods used in similarity-based query systems. Section 5 proposes a model of image data repository. Section 6 introduces the novel similarity-based algebraic operators and studies their properties. Section 7 shows the way that our image data repository model can be extended to support similarity-based operations on salient objects. In Section 8, a query example in a medical application is given and the significance of our similarity-based operators is discussed shortly. Based on the novel operators introduced, Section 9 presents similarity-based Query optimization issues. Finally conclusion is given in Section 10.

2 Motivation

To demonstrate the practical significance of our work we present below some example queries. Let an *image table* consist of the attribute components: image, the feature vector representation of the image, a component that contains alphanumeric information about the image, and some other components. Let EMP and SI be two image tables. EMP is a table of employee of a company and contains the attribute components: the photo of the employee, the feature vector representation of the photo, the name and address of the employee, his/her department, and his/her occupation. Let SI contain images of individuals who appeared and entered by the front gate of the company where a surveillance camera is mounted. Let the components of SI be: the images taken by the camera, the corresponding feature vector representations, the date and time at which each of the images was scanned or taken.

Suppose now that there is an investigation scenario of an event that is associated to the images in SI. SI alone cannot give complete information of a person. It is therefore necessary to perform some operations on the tables in order to get complete information of a person whose image is in SI. It is important to note that operations based on similarity measures do not perform accurate matching. It is therefore a common practice to search for a range of the most similar images and then browse among the selected images. The following are possible queries.

1) *For a particular picture of a person in SI, find its most similar photos that are in EMP.*

This query is what most of the currently available content-based image retrieval systems support.

Suppose now that we are interested only in the employees of a certain department (e.g "Production" department). Then, the following is a possible query.

2) *For the pictures in SI, find their corresponding most similar photos that are in EMP and where the persons are in the "Production" department.*

This query requires a "similarity-based join" operation (i.e. a matching between the feature vector representations of the photos of SI and EMP), and a relational selection on the attribute, department. To the best of our knowledge, existing image query systems does not support this kind of query.

If we are interested in the detailed information of the individuals whose images were scanned on a particular date, say "yesterday from 4 to 6PM", we will be required to perform a selection operation on the time and date attributes of SI and then a similarity-based join operation on the image representation components of SI and EMP. A possible query for this is:

3) *For pictures of individuals in SI that were scanned on September 11 from 6 to 8AM, find their corresponding most similar images from EMP, with their corresponding name and address.*

Processing this query requires a relational selection on the SI table and a "similarity-based join" on SI and EMP. So we need to be able to combine and process relational and similarity-based operations on image tables.

Suppose now that the surveillance system is organized in such a way that there is one camera mounted at the external gate of the company's compound and another camera at the internal gate of a particular department. Let SIE be the image table created at the exterior gate and SII be the image table at the interior gate. The following is a possible query for a particular investigation scenario.

4) *Get the images and the names of employees who entered the gate of the department within two hours after they entered the main gate of the company on a particular date.*

This query requires among others a similarity-based join on the three image tables SIE, SII, and EMP by the feature vector components of the images.

The relational selection and join operations have been used efficiently for more than three decades and their properties are well-studied. On the other hand, similarity-based operations are not yet widely exploited. The examples above clearly illustrate the need for a comprehensive and efficient use of the two kinds of operations in combination. In this perspective, the study and the introduction of a similarity-based algebra that will allow formalizing content-based operations is a key issue if one wants to be able to combine traditional and content-based operators into complex queries and to optimize and process these queries in an efficient way. The goal of this paper is to propose and formulate such an algebraic framework.

3 Related Work

In the domain of visual pattern recognition, a lot of work has been done for an effective representation of visual data such as image and video by its low-level features of color, texture, shape, etc. [7].

These low-level features can then be used for the purpose of visual recognition and content-based retrieval of the image and video data. Using this technique, a number of research prototypes and commercial content-based image retrieval systems such as Photobook, Netra, Surfimage, VisualSeek, CAFIIR, STAR etc. [19, 27, 11, 9, 18] are currently in practice. Commercial database

management systems have also started to integrate content-based retrieval modules in their systems to support image data. QBIC is a content-based image query system available commercially either in stand alone form, or as part of other IBM products such as the DB2 Digital Library. It offers retrieval by any combination of color, texture or shape - as well as by text keyword [27]. However, it doesn't support operations such as the "similarity-based join". The VIR Image Engine from Virage is a well-known commercial system. It is available as a series of independent modules that system developers can integrate in to their own programs. The engine provides the fundamental capability for analyzing and comparing images for similarity. But, it has no concept of persistent storage, user interfaces, query processing nor optimization [9]. It is a pure set of algorithms that is designed to be embedded in software and hardware systems where its capabilities are needed. This makes it easy to be extended to process specialized collections of images by building new types of query interface or additional customized modules. As a result, the VIR Image Engine is available as an add-on to the existing database management systems such as Oracle8i and 9i Enterprise Editions [17]. The Excalibur Image DataBlade Module from Excalibur Technologies is a product that offers a variety of image indexing and matching techniques based on the company's own proprietary pattern recognition technology. This datablade module is incorporated in the Informix database system to support a content-based image storage and retrieval [16]. A common feature of the add-on modules integrated in these DBMSs is that, given a query image, they search its most similar images from a database of images using their respective content-based image retrieval engines. That is, the attempts so far didn't exceed from these one-to-many content-based image retrieval operations. A positive result of these works is that, one can use SQL based statements to store, and retrieve images using content-based feature representations. However, as said before, these systems are strongly limited in terms of supporting complex similarity-based operations and mixed queries involving both relational and similarity-based operations.

One of the recently completed project for a compound retrieval of both content- and semantic-based representations of images is the DISIMA system of University of Alberta [15]. DISIMA has demonstrated well a mechanism of content- and semantic-based image retrieval and extended the OQL standard for a Multimedia Object Query Language (MOQL). The project has also developed a visual user interface, Visual MOQL. However, DISMA also lacks to show mechanisms of similarity-based join operations and other complex operations.

Another important area of research in database systems is Query optimization. Query Optimization is the process of choosing the optimal processing strategy to answer a query. Relational query optimization is still an active research issue after more than twenty years of research and experience in the field [12, 23]. When it comes to image database systems, query optimization is a fresh research area. Indeed, most of the currently existing similarity-based retrieval systems focus mainly on their content-based retrieval engine input/output capabilities and gave less emphasis to content-based query optimization. Actually, one of the key issues, as stated by S. Adali et al. [1] is that, there has been no work on developing an algebra for similarity-based database query operations. However, the "multi-similarity algebra" that these authors propose remains at a higher abstraction level. That is, it does not address the definition of an "operational" similarity-based algebra usable for modeling, optimizing and processing *similarity-based queries* combining similarity-based and relational operators at the implementation level.

In this paper, we will show that the system of similarity-based operators we introduce will address the fundamental issue to create a similarity-based algebraic space useful for an integrated query system and which is also useful to formulate basic rules for a system of similarity-based query optimization.

4 Similarity-Based Query

In this section, we summarize the methods used for similarity-based searching and prepare the ground to propose our schematic model for storing image data and to define our novel similarity-based algebraic operators.

Given an image query object, similarity-based searching retrieves an object or set of image objects that is/are similar to the query object, from a larger set of image objects using their feature vector representations. The feature vector representations of the query object and the set of objects to be searched for similarity are required to have the same dimension and are formed with identical feature measures.

Similarity-based query systems thus use feature vector representations of the images for the purpose of similarity matching. First, the feature vector representations of the images are mapped as data points in an n-dimensional feature space. Then, the similarity comparison is performed based on distance metrics. A commonly used method to determine similarity between images is the method called *k-Nearest Neighbor* (k-NN) search [4, 3, 21]. Using the same principle however, a *range query* is also widely practiced [13]. In both approaches, the similarity of images is determined by the proximity of the data points representing the images in an n-dimensional feature space. Based on this, in the remaining part of this paper, we will identify the "distance" between two images as the distance between the data points representing the images in the n-dimensional feature space. A similarity-based search uses a notion of non-exact matching. What is then commonly practiced is that, one searches for a certain number of similar objects and then browses interactively to identify the most matching ones. For the purpose of reference and comparison, we define both k-NN and range query below.

Definition 4.1. (k-Nearest Neighbor Search)
Given a set of images S, a query image q, and a positive integer k; the k-nearest neighbors to the query image q denoted as $NN_k(S,q)$ are the first k images that are at the closest distance from q in the feature space than any of the other images in S. More formally:
$NN_k(S,q) = \{\{o'_1, o'_2, ..., o'_k\} | o'_i \in S \; for \; i = 1, ..., k \; and \; \|o'_i - q\| \leq \|o - q\| \; \forall o \in S \backslash \{o'_1, o'_2, ..., o'_k\}\}$, where $\|o - q\|$ stands for the distance between the object o and the query object q.

The value of k is assumed to be less than the cardinality of S and at the instance where there are more than one image at the kth closest point to q, the k-NN search algorithm selects one randomly. Nearest Neighbor search algorithms are discussed in [4, 3, 21, 10]. For k=1, it searches for the most similar object.

In similarity-based range query, all the similar objects to a given query object that are within a given range distance ε are retrieved. A formal definition of similarity-based range query is given below.

Definition 4.2. (Similarity-Based Range Query)

Let S be a set of images, q be a query image, and ε be a positive real distance value. A similarity-based range query of q with respect to S and ε, denoted by $R^{\varepsilon}(S,q)$, is a similarity-based query that returns all image data points of S that are within distance ε from the query image q. Formally, it is given as:
$R^{\varepsilon}(S,q) = \{o' \in S | \; \|o' - q\| \leq \varepsilon\}$, where $\|o' - q\|$ denotes the distance between o' and q.

The range query returns an unspecified number of objects (from zero to some number r) that are similar to q. So, the value of ε must be selected in an optimal manner that considers both the user's needs and the data point density in the feature space. Often, the value of ε is selected by the user. Range query is discussed in [13].

We defined above the two most widely used methods for content-based image retrieval. However, we will use the range query method to define our similarity-based operators. The reason why we choose the range query is that, the range query posses better properties that can be used for similarity-based query optimization. These properties are widely discussed in Sections 6 and 9. Issues in selecting an optimal value for ε is discussed in Section 8.

5 Image Data Repository Model

In a relational table, each attribute of a table describes the relation or the instance of an entity using alphanumeric information. Operations on such a table use direct attribute value based matching or comparison on numeric values or text strings. A table that contains an image as an attribute is quite different from the traditional tables. Because, the later requires other attributes that describe the image attribute either using keywords or its low-level features. So, attributes are used not only to describe the instance but also to describe the image attribute of the table since images possess much information that needs to be described in more details. Furthermore, when a table consists of a content-based representation of an image, the database system requires different techniques to store, describe, and manipulate, the image data than that of the relational systems. Hence, an image data repository should be able to capture the low-level feature descriptions of the image data in addition to the metadata representation for higher level semantic features of the images. An image data repository at least needs the following additional requirements as compared to the relational tables.

1. a table with image data require a much larger storage requirement, which has a direct impact on the data management,
2. the content of image data must further be described within the record mostly using both descriptions of low-level features and metadata representations and that these descriptions require the introduction of new data structures in the table definition,
3. it consists of image objects that require complex data structures to handle. The introduction of these new data structures, the image storage, the low-level feature descriptions of the images, and the image data query techniques introduce new types of operations (or similarity-based operations) and methods of data management, and
4. an image data repository model, should consider the constraint of large data storage requirements during intermediate results of similarity-based operations.

Hence, an image data repository model should be designed in such a way that it manages all these characteristics of an image data. In this paper, we propose a novel image data repository model (which we also refer it as *image table*), under an OR paradigm. M. Stonebraker et al. [23] have widely elaborated in saying that the next wave of database management systems is under the OR paradigm. The OR paradigm is a system that conveniently support image data in a DBMS. So, we keep our discussion within this framework.

Definition 5.1. (Image Table)
An object-relational image table is defined as a table of five components $M(id, O, F, A, P)$. Where:

- id is a unique identifier of an instance of M,
- O is a reference to the image object itself which can be stored as a $BLOB$ internally in the table or which can be referenced as an external $BFILE$ (binary file),
- F is a feature vector representation of the object O.
- A is an attribute component that may be used to describe the object using key-word like annotations. A may be declared as an object- or a set of object types,
- P is a data structure that is used to capture pointer links to instances of other tables associated by a binary operation.

Note that, in an image table M, the item of primary importance is the image itself (i.e. the remaining attributes are associated to the image). The three components O, F, and A can be used to capture sufficient information on an image data. P can be considered as a column whose content is a data structure that can store links to instances of other tables during binary operations such as similarity-based join. P has a value $null$ in the base tables. It has a non-null value in intermediate tables during binary operations. More formally, when it contains a value, P can be expressed as a set of tuples of the form ($table$, set_of_ids), where the component $table$ denotes the associated table by a binary operation and set_of_ids is the set of its referenced id elements. After a similarity-based binary operation like a similarity-based join of M_1 and M_2, let the P component of the resulting image table M' holds a link to a table M_2. Then, for each instance of M', P will contain elements of the form $(M_2, \{id_2^1, ..., id_2^h\})$, where h is the number of instances of M_2 associated to M_1 by the similarity-based binary operation.

With the help of this *image table* model, we can manage the necessary requirements to integrate a content-based image data management under an ORDBMS effectively.

6 The Similarity-Based Operators

The example queries in Section 2 illustrate the need to have some basic similarity-based operators and a heuristic to use these similarity-based operators in combination with the traditional operators. In this section, we define several novel operators that are required for an image management in a content-based image database system. We first give the definitions of the basic similarity-based operators. Then, we will study their properties and see the way we can use them in conjunction with the traditional database operators.

To define the novel similarity-based operators on image tables we use the method of content-based range query [20]. A content-based range query on a set of images S returns those image objects that are within distance[3] of ε from the query image q. Thus, we define our similarity-based operators as given below. At the end of this section, we will show that the properties of the range query is advantageous over the k-Nearest Neighbor method [21], particularly for similarity-based query optimization in image database systems.

6.1 The Similarity-Based Selection Operator

The similarity-based selection operator is a unary operator on an image table performed on the component F based on the following definition.

[3] *Here, distance is defined as the distance between the feature vectors representing the images in a feature space*

Definition 6.1. (Similarity-Based Selection)
Given an image query object o with its feature vector representation, an image table M, and a positive real number ε; a similarity-based selection operation denoted by $\sigma_o^\varepsilon(M)$, selects all the instances of M whose image objects are most similar to the query image object o. Formally it is given by:
$\sigma_o^\varepsilon(M) = \{(id, o', f, a, p) \in M | o' \in R^\varepsilon(M, o)\}$, where $R^\varepsilon(M, o)$ denotes the range query with respect to ε for query image object o and a set of images in the image table M.

The similarity-based selection operation first uses the range query search method to select the image objects that are most similar to o from the objects in M (which is also expressed by the notation $R^\varepsilon(M, o)$). Then, it identifies the instances of M whose image objects are selected to be similar to o.

6.2 The Similarity-Based Join Operator

A similarity-based join is a binary operator on image tables performed on the feature vector components as defined formally below.

Definition 6.2. (Similarity-Based Join Operation)
Let $M_1(id_1, O_1, F_1, A_1, P_1)$ and $M_2(id_2, O_2, F_2, A_2, P_2)$ be two image tables and let ε be a positive real number. The similarity-based join operation on M_1 and M_2, denoted by $M_1 \otimes^\varepsilon M_2$, associates each object O of M_1 to a set of similar objects in M_2 with respect to the F components of M_1 and M_2. The resulting table consists of the instances of M_1 (the table at the left) where P is modified by inserting a pointer pointing to the set of the associated instances of M_2 (the table at the right side of the operation). Formally it is given by:
$M_1 \otimes^\varepsilon M_2 = \{(id_1, o_1, f_1, a_1, p_1') | (id_1, o_1, f_1, a_1, p_1) \in M_1 \; and \; p_1' = p_1 \cup (M_2, s_id^\varepsilon(M_2, o_1)) \; and \; p_1' \neq null\}$, where $s_id^\varepsilon(M_2, o_1) = \prod_{M_2.id}(\sigma_{o_1}^\varepsilon(M_2))$ (i.e. the *ids* contained by the projection on the *id* component of the associated instances of M_2).

Query 3 of Section 2 requires an operator of this type. As stated in the definition, for a similarity-based join $M_1 \otimes^\varepsilon M_2$ the components of P_1 can be expressed as $(M_2, s_id^\varepsilon(M_2, o_1))$, where $s_id^\varepsilon(M_2, o_1) = \prod_{M_2.id}(\sigma_{o_1}^\varepsilon(M_2))$. We will use this form of expression for the elements of P in the rest part of this paper. Here, we may require a function which allows us to retrieve the instances of M by their unique identifier components. So, we define $get_instance(M, query_id) = (query_id, o, f, a, p)$, such that $(query_id, o, f, a, p) \in M$. Actually, the *get_instance* function is a relational selection operation on the *id* component of M. We will use this *get_instance* function latter in Section 6 for the "Mine" algorithm.

Identical to Definition 6.2 and considering the properties of similarity-based matching between images, we define below the way we compute a *multi similarity-based join* (See Figure 1 for the join $M_1 \otimes^\varepsilon M_2 \otimes^\varepsilon M_3$). To facilitate the defination of this operator, let us first define a basic operator called, the *Additive Union*.

Definition 6.3. (Additive Union)
Let M_1 and M_2 be two image tables, the Additive Union of M_1 and M_2 denoted by $M_1 \uplus M_2$ is an image table that merges all the records of M_2 to M_1 or vice versa. The additive union contains all the instances that are either in M_1 or in M_2, without excluding none of the instances of M_1 or M_2.

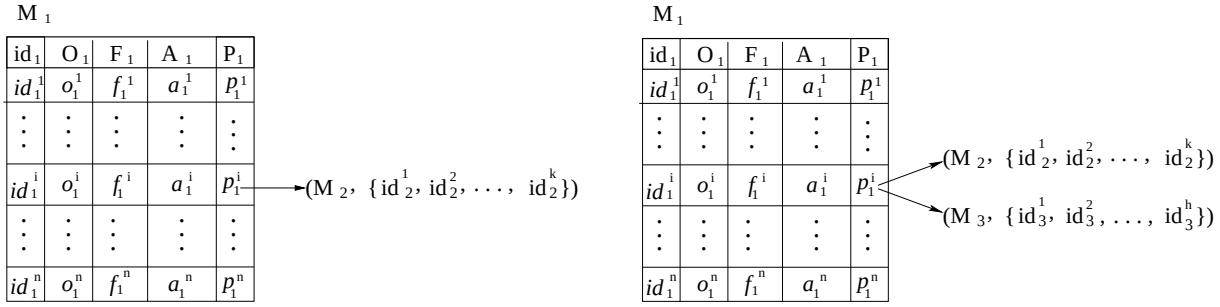


Figure 1: The Similarity-based joins $M_1 \otimes^{\varepsilon} M_2$ and $M_1 \otimes^{\varepsilon} M_2 \otimes^{\varepsilon} M_3$ respectively

.

Definition 6.4(The Multi Similarity-Based Join)
Let M_1, M_2, ... M_n be n image tables. The multi similarity-based join denoted by $M_1 \otimes M_2 ... \otimes M_n$ is defined as:

$$M_1 \otimes M_2 \otimes ... \otimes M_n = \biguplus_{i<j;\ \ 1 \leq i,\ j \leq n} M_i \otimes M_j$$

Note that, here, the order of the operand tables is important or has a semantic interpretation. The additive union operator ($\uplus$) is used just for the purpose of stocking together the resulting tables of each of the pair wise similarity-based joins.

Each instance of an image table and a resulting table of a similarity-based join is identified by its unique identifier, *id*. Note that if the content of p_1 is *null* (not pointing to M_2), then its instance will not be contained in the resulting table. This definition of a similarity-based join reflects the practical needs of content-based image retrieval. Figure 1. illustrates, the resulting tables of two similarity-based joins. The similarity-based operators defined above depend on "relative" measures. Due to this, the similarity-based operators possess different algebraic properties than that of the relational ones. For example, contrary to the relational join operator, we observed that the similarity-based join operator $\otimes^{\varepsilon}$ is not commutative. This shows that in similarity-based join operation, the order of the image tables is meaningful. If an image table appears at the left of a similarity-based join, then its objects are taken as reference for the similarity operation. In the example Query 3 of Section 2, the image table SI should be our reference table and hence should appear at the left if we are interested to know about the individuals who entered the gate. Furthermore, in the similarity-based join operator $\otimes^{\varepsilon}$ we do not use the concept of associativity. Due to the non-transitive property of similarity-based matching on image tables, we don't use intermediate tables as operands. Our definitions confirms with this property of similarity-based matching. It is important to note that, an operator without the properties of commutativity and associativity is difficult to be exploited for query optimization. We thus need to see possibilities of extending this operators, so that it posses useful properties for query optimization.

6.3 A Symmetric Similarity-Based Join operator

In view of the current practices of content-based query systems (where for a given image query object, we search for its most similar objects from a database of image objects), the similarity-based join operator defined above is what may be needed for many applications. However, it doesn't posses a symmetric property. This makes it lose one of the major good properties that the relational join operator has and that is particularly important to query optimization. To make the

similarity-based join operator defined above suitable for similarity-based query optimization, we extend it to a *Symmetric Similarity-Based Join* operator. As far as our knowledge is concerned, there is no effort done so far in studying such properties of similarity-based operations on images.

To perform a similarity-based binary operation on two image tables, we assume that their feature vector component F are extracted identically in such a way that it permits a meaningful computation of range query. Moreover, when we perform additive union, if the A components do not have the same structure we take the union of the A components of the two operand image tables. Here it is important to note that additive union is commutative. Below we define a symmetric similarity-based join that makes use of the similarity-based join operation and the additive union operator.

Definition 6.5. (Symmetric Similarity-Based Join)
Let M_1 and M_2 be two image tables, the symmetric similarity-based join of M_1 and M_2 denoted by $M_1 \oplus^{\varepsilon} M_2$ is formally defined as:

$$M_1 \oplus^{\varepsilon} M_2 = (M_1 \otimes^{\varepsilon} M_2) \uplus (M_2 \otimes^{\varepsilon} M_1).$$

From this definition we see that the symmetric similarity-based join operator posses the following property.

Property 1: The symmetric similarity-based join operator $\oplus^{\varepsilon}$ is commutative. i.e. $M_1 \oplus^{\varepsilon} M_2 = M_2 \oplus^{\varepsilon} M_1$.

This follows directly from the commutativity of the additive union $M_1 \uplus M_2$. Figure 2 illustrates the way the symmetric similarity-based join of two image tables is computed.

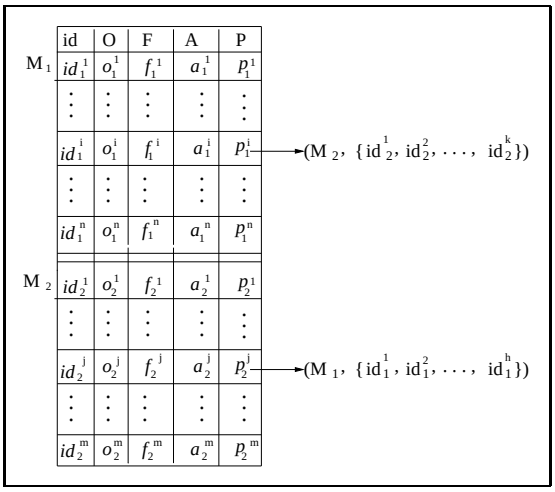


Figure 2: The symmetric similarity-based join $M_1 \oplus^{\varepsilon} M_2$.

We can now generalize the symmetric similarity-based join on more than two image tables and define a *Symmetric Multi Similarity-Based Join*. This definition reflects the characteristics of similarity-based operations and maintains useful properties for similarity-based query optimization.

Definition 6.6.(The Symmetric Multi Similarity-Based Join)
Let M_1, M_2, ... M_n be n image tables. The Symmetric Multi Similarity-Based Join, denoted by

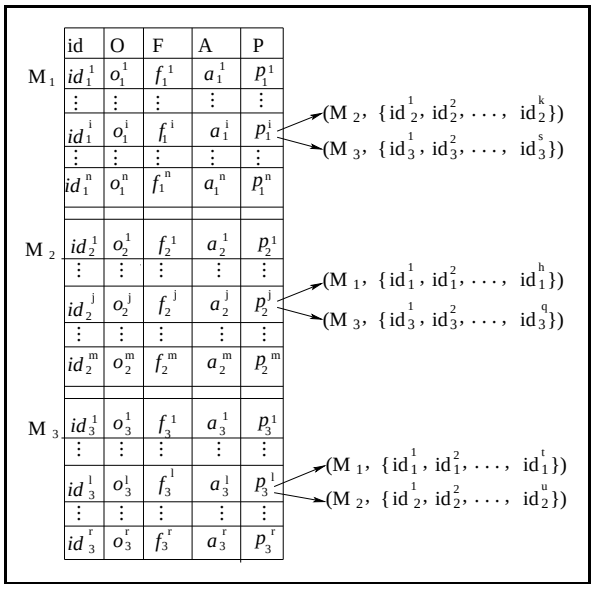


Figure 3: The symmetric similarity-based join: $M_1 \oplus^{\varepsilon} M_2 \oplus^{\varepsilon} M_3$.

$M_1 \oplus M_2 \oplus ... \oplus M_n$ is defined as:

$$M_1 \oplus M_2 \oplus ... \oplus M_n = \biguplus_{i<j;\ \ 1 \le i,\ j \le n} M_i \oplus M_j$$

The way the symmetric multi similarity-based join operates is not identical to that of the multiway join known in relational database systems. Here, it doesn't consider intermediate resulting tables as operands. Figure 3 illustrates how the symmetric multi similarity-based join operates on three image tables. This depicts that, the following property holds true.

Property 2: The order of the operand tables is not relevant on the result of a symmetric multi similarity-based join.

This property could rather be useful when considering the properties of content-based image retrieval. It is important to note here the order of the operands is not relevant on the result of a multi symmetric similarity-based join. Though order is not relevant for the result, the order of each pair of operands can effectively be utilized for query optimization purpose as in the case of relational join operation. That is it enables us to formulate newer rules for query optimization.

6.4 Relationships Between the Symmetric and Non-Symmetric Similarity-Based Joins

6.4.1 The *Extract* Operator:

The Similarity-Based Join, stated in Definition 6.2. (that is non-symmetric), can be expressed in terms of the symmetric similarity-based join by the use of an operator called *Extract*. Hence, the following holds true.

$M_1 \otimes^{\varepsilon} M_2 = Extract_{M_1}(M_1 \oplus^{\varepsilon} M_2)$ and
$M_2 \otimes^{\varepsilon} M_1 = Extract_{M_2}(M_1 \oplus^{\varepsilon} M_2)$,
where the $Extract_{M_1}(M_1 \oplus^{\varepsilon} M_2)$ extracts the instances of M_1 from $M_1 \oplus^{\varepsilon} M_2$ keeping the modified

P component of M_1. Similarly, $Extract_{M_2}(M_1 \oplus^{\varepsilon} M_2)$ extracts the instances of M_2 from $M_1 \oplus^{\varepsilon} M_2$ keeping the modified P component of M_2. Similarly, the *Extract* operation can be applied on *Symmetric multi similarity-based joins* to get any of the non-symmetric *multi similarity-based joins*. For example,

$$M_1 \otimes^{\varepsilon} M_2 \otimes^{\varepsilon} M_3 = Extract_{M_1}(M_1 \oplus^{\varepsilon} M_2 \oplus^{\varepsilon} M_3)$$

6.4.2 The *Mine* Operator:

We stated above that the symmetric similarity-based join could be interpreted as the additive union of the two non-symmetric similarity-based joins.

After having the resulting table for one of the non-symmetric joins, we present here the method to get the other without applying similarity-based operations. If for example we have the resulting table of $M_1 \otimes^{\varepsilon} M_2$ with its new P_1 component, we can make use of the contents of P_1 to get the instances of $M_2 \otimes^{\varepsilon} M_1$ by direct statistics collection. This follows from the idea that, given the same value of ε, if an object o_2 of M_2 is within a distance of ε to the object o_1 of M_1, then the converse is also true, by the symmetric property of distance. This is a property that we can get from the use of range query [20] (i.e. we can not have this advantage if we use for example the k-NN search method). Hence, we can compute the symmetric similarity-based join, $M_1 \oplus^{\varepsilon} M_2$, by performing only one of the non-symmetric joins and then using an operator called the $Mine$ operator for the other. The major advantage of this approach is that, the $Mine$ operator could be much less expensive than similarity-based join. Thus, it can be used for query optimization purpose for both the symmetric and non-symmetric similarity-based join operations.

Let us first define and demonstrate the use of the $Mine$ operator on a simple similarity-based join, $M_1 \otimes^{\varepsilon} M_2$. Then, we will show how it can be generalized for any complex similarity-based join expression.

Definition 6.7.(The Mine operator)
Consider the similarity-based join $M_1 \otimes^{\varepsilon} M_2$. The $Mine$ operator on $M_1 \otimes^{\varepsilon} M_2$, denoted as $Mine(M_1 \otimes^{\varepsilon} M_2)$ uses the component P of the resulting table of $M_1 \otimes^{\varepsilon} M_2$ and builds the table $M_2 \otimes^{\varepsilon} M_1$. Conversely, $Mine(M_2 \otimes^{\varepsilon} M_1)$ uses the component P of the resulting table of $M_2 \otimes^{\varepsilon} M_1$ and builds the table $M_1 \otimes^{\varepsilon} M_2$.

Let $M_1(id_1, O_1, F_1, A_1, P_1)$ and $M_2(id_2, O_2, F_2, A_2, P_2)$ be two image tables. Suppose we want to compute $M_1 \oplus M_2$. Let us first consider that we are computing $M_1 \otimes M_2$. Figure 3 shows the projection of the image data points of M_1 and M_2 on a 2D plane for illustration purpose. As shown on the figure, for the image object o_1^i of M_1 and a given value of ε, the content-based range query looks for all objects o_2 of M_2 that are within a distance of ε. Thus, it selects the image data points: o_2^t, o_2^u, o_2^v, and o_2^w of M_2. Then, the similarity-based join $M_1 \otimes M_2$ identifies the id's of the instances of M_2 containing these objects are stored in the corresponding P component of M_1 in the resulting table. From this example we clearly see that, when we latter on want to compute $M_2 \otimes M_1$, o_1^i of M_1 will be returned as an image object within a range of ε for all objects of o_2^t, o_2^u, o_2^v, and o_2^w of M_2, by the symmetric property of distance. Since this information is already contained in the P component of the instance of $M_1 \otimes M_2$ that contains the object o_1^i, we can get this information from $M_1 \otimes M_2$. This can be repeated on all P's of $M_1 \otimes M_2$ to get all the instances of $M_2 \otimes M_1$.

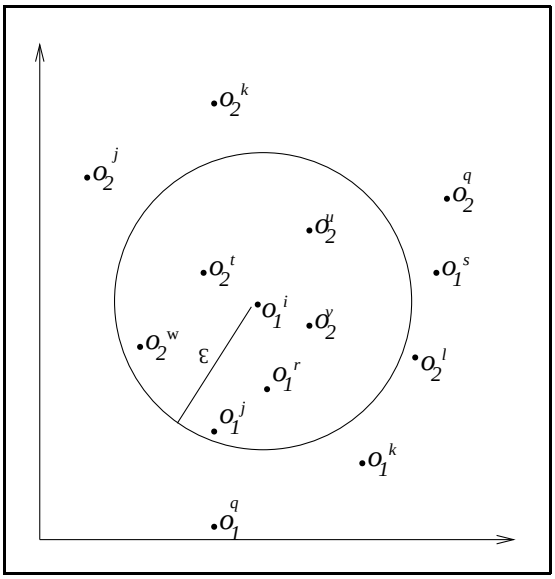


Figure 4: Illustration for the *Mine* operator.

To demonstrate the use of *Mine* let us first distinguish the difference between *non-referring* and *non-referenced* instances of an operand image table of a similarity-based join. For the similarity-based join $M_1 \otimes^{\varepsilon} M_2$:

- if the object of an instance of M_1 has no similarity matching in M_2 by the similarity-based join, we say that it is a non-referring instance. Conversely,
- if an instance of M_2 is not associated by the similarity matching of the similarity-based join, we say that this instance is a non-referenced instance of M_2.

Considering the similarity-based join $M_1 \otimes^{\varepsilon} M_2$, suppose M_1 has a set of referring instances M_1' and non-referring instances $M"_1$. Note that, M_1' and $M"_1$ are disjoint and that $M_1 = M_1' \uplus M"_1$. Suppose also that M_2 has a set of referenced instances M_2' and a set of non-referenced instances $M"_2$. Note also that, M_2' and $M"_2$ are disjoint and that $M_2 = M_2' \uplus M"_2$. Then, the symmetric similarity-based join expression: $M_1 \oplus^{\varepsilon} M_2 = (M_1' \uplus M"_1) \oplus (M_2' \uplus M"_2)$ will finally be reduced to: $M_1 \oplus^{\varepsilon} M_2 = (M_1' \otimes^{\varepsilon} M_2') \uplus (M_2' \otimes^{\varepsilon} M_1')$.

Thus, if $M_1' \otimes^{\varepsilon} M_2'$ is computed using the similarity-based join operation of Definition. 4.2, the other (i.e. $M_2' \otimes^{\varepsilon} M_1'$) can be generated using the *Mine* operator or vice-versa. The Algorithm on Figure 2 demonstrates how the *Mine* operator computes $M_2' \otimes^{\varepsilon} M_1'$, from the resulting table of $M_1' \otimes^{\varepsilon} M_2'$. Note that, if an instance of M_1 is not referring in the join $M_1 \otimes^{\varepsilon} M_2$, then it can not be re-referenced by $M_2 \otimes^{\varepsilon} M_1$. Furthermore, dropping the non-referring and non-referenced instances from the resulting image table will not affect the commutative property of the symmetric similarity-based join.

There is a great deal of work that can be done by a query optimizer to select the appropriate strategy. It is up to the query optimizer to choose which similarity-based join to do first and then use the *Mine* operator for the other. Hence, we can clearly see that the symmetric similarity-based join can be more effectively processed with query optimization techniques.

The *Mine* operator given above for a simple similarity-based join (i.e. a similarity-based join between two image tables) can be generalized to apply on complex similarity-based join expressions. The general principle is that, for each one-sided similarity-based join, we can compute the other sided join with the use of the *Mine* operator.

Create table T = T(id, O, F, A, P)
Foreach instance $inst$ of M' **Do**
    Foreach element id_2 of $inst.P'_1$ **Do**
        If id_2 is not in T
        Append $get_instance(M_2, id_2)$ in T
        End If
        Update P of $T.id_2$ with $M'.id_1$
    End Do
End Do
Return(T). /* $T = M_2 \otimes^{\varepsilon} M_1$ */

Figure 5: Algorithm for the $Mine$ operator

For a short description, let us consider the symmetric similarity-based join of three image tables M_1, M_2, and M_3, $((M_1 \oplus^{\varepsilon} M_2) \oplus^{\varepsilon} M_3)$. To compute $M_1 \oplus^{\varepsilon} M_2 \oplus^{\varepsilon} M_3$, the system needs to process six similarity-based joins (non-symmetric) based on Definition 4.2. However, we can only compute three of the similarity-based joins and the rest three can be generated using the $Mine$ operator. In like manner, to compute a multi symmetric similarity-based join with n operand tables, we need to compute $n(n-1)$ non-symmetric similarity-based joins, out of which $n(n-1)/2$ of them can be generated using the $Mine$ operator. Then, the results can be merged to form the multi symmetric similarity-based join based on Definition 4.5.

6.5 Other Operators on Image Tables

In relational database systems, the Intersection, Union, and Difference operators are among those operators that are commonly used. These operators can be adopted to be used on image tables of the schematic form $M(id, O, F, A, P)$ we considered in this paper. Below, we describe, the way that we can perform these operations on image tables.

- The **Intersection** of two image tables M_1 and M_2, denoted by $M_1 \cap M_2$, is the set of instances that are in both M_1 and M_2. M_1 and M_2 are required to have the same attributes. Comparison on the A component is based on the relational rules. We say that two F values are the same, if their feature vector representation match when placed in the feature space with a certain freedom of error, as far as this error doesn't have effect on the identicalness of the other components of the instances of the two tables. This error could be an error that emanate as an error during transforming the image to its feature vector representation. The F components of M_1 and M_2 must be created with the same feature measures and the A components need to have the same attribute elements.

- The **Union** of two image tables denoted as $M_1 \cup M_2$, is the set of instances that are in M_1 or M_2 or both. Comparison on the F and A components is done as discussed in the case of the intersection operation above.

- The **Difference** of two image tables M_1 and M_2, denoted as $M_1 - M_2$, is the set of instances in M_1 but not in M_2.

A Cartesian product of two image tables may be defined as follows.

Definition 6.8. (Cartesian Product)
The Cartesian product of two image tables M_1 and M_2 denoted by $M_1 \times M_2$ is defined ad:
$M_1 \times M_2 = \{(id_1, o_1, f_1, a_1, p_1') | (id_1, o_1, f_1, a_1, p_1) \in M_1 \; and \; p_1' = (M_2, set \; of \; all \; ids \; of M_2)\}$.

The use of the operators discussed in this section, will permit us to create a system of similarity-based algebraic operations on image databases, identical to the relational algebraic systems in RDBMS.

6.6 Relational Operators on Image Tables and Complex Expressions

It is a practical requirement to define a relational join of two image tables (for example on the elements of the A components). A relational operation on image tables is performed based on the rules of relational algebra. One of our goals other than the semantic correctness in image database similarity-based operations is to avoid having two images in the resulting table. So, in the same manner to the similarity-based join, we can use the P component of one table to store pointer structures to the joined instance(s) of the other table(s) in a relational join of image tables. However, in the case of a relational join of image tables, the operation is based on the non-image components. We can therefore keep the joined components in one of the image table and point to the other using the P component. This leads us to the choice of performing either a left Relational Join (lR-Join) or a right Relational Join (rR-Join) operation on two image tables. The choice could be based on the requirement for a subsequent operation. A lR-Join on two image tables performs a relational join based on a join condition jc and retains the instances of the table at the left while modifying its P component by inserting a pointer to the joined instances of the table at the right. Respectively, a rR-Join retains the instances of the table at the right while modifying its P component by pointing to the joined instances of the table at the left. The formal definitions are given below.

Definition 6.9. (left Relational Join)
Let M_1 and M_2 be two image tables. The left relational join of M_1 to M_2 under the join condition jc is defined formally as:
$M_1 \bowtie^l_{jc} M_2 = \{(id_1, o_1, f_1, a_1, p_1') | t = (id_1, o_1, f_1, a_1, p_1)$
$\in M_1, p_1' = p_1 \cup (M_2, R(t)), R(t) \neq \emptyset\}$, with $R(t) = \{t'.id, t' \in M_2 | jc(t, t')\}$, where $jc(t, t')$ with some $t \in M_1 \; and \; t' \in M_2$ is a predicate which holds true when the join condition jc is satisfied.

Definition 6.10. (right Relational Join)
Let M_1 and M_2 be two image tables. The right relational join of M_1 to M_2 under the join condition jc is defined formally as:
$M_1 \bowtie^r_{jc} M_2 = \{(id_2, o_2, f_2, a_2, p_2') | t = (id_2, o_2, f_2, a_2, p_2) \in M_2, p_2' = p_2 \cup (M_1, L(t)), L(t) \neq \emptyset\}$,with $L(t) = \{t'.id, t' \in M_1 | jc(t', t)\}$, where $jc(t', t)$ with some $t' \in M_1, t \in M_2$ is a predicate which holds if the join condition jc returns true.

A join between an image table and a relational table may also be required in many practical applications. Thus, we define a relational join operator between an image table and a relational table as follows.

Definition 6.11. (A Join between an Image- and a Relational Table)
Let R be a relational table and $M(id, O, F, A, P)$ be an image table. A relational join between M and R will result in a modified image table M' which integrates R in the A component of M based on the join condition jc. Formally: $M' = R \bowtie_{jc} M = M \bowtie_{jc} R = \{(id, o, f, a', p) | t = (id, o, f, a, p) \in M, a' = (a, t'), t' \in R, jc(t, t')\}$.

This definition can as well be applied to perform a relational join of an image table to an alphanumeric object-relational table. Furthermore, using the same approach, the *id* component of M may be considered as an attribute for a relational join with other relational table., if it is found to be a meaningful operation. Note that in the case of a join between an image table and a relational table, there will be no modification on the P component of M. The Cartesian product between a relational table R and an image table M can be defined similarly.

7 Supporting Operations on Salient Objects

An image is a complex item that may contain different objects of interest. These particular meaningful objects of interest in an image are referred in the literature as *Salient objects* [25]. In some applications, a content-based retrieval requires to consider the salient-objects of an image. Hence, Image Database systems should enable us to operate on these particular objects of an image. In current practices, content-based image retrieval systems have shown better effectiveness when supporting salient objects for image similarity-based comparisons [11]. Effective similarity-based retrieval engines for applications such as facial image retrieval, medical images retrieval, etc. use the concept of segmentation [11, 26]. Segmentation is a technique that deals with the process of identifying the objects of interest (or salient objects) within an image. The integration of segmented objects in similarity-based image retrieval is an important issue and will inevitably be supported by most of the future content-based multimedia data retrieval systems.

Our model for an object-relational image table, $M(id, O, F, A, P)$, is designed in such a way that it can be extended to support the management of salient image data objects. Thus, operations on salient objects could be made possible. Suppose that the low-level feature representations of the salient objects are extracted, and the semantic features of the corresponding salient objects are described. Then, we can capture all information related to the salient object with a data repository model: $S(id_s, F_s, A_s)$. Since an image may have more than one salient objects, we identify each of the objects a unique identifier, id_s. F_s is the feature vector that represent the low-level content of a salient object, and A_s is an object attribute that can be used to capture all its semantic features.

Since a salient object is part of an image (i.e. not a separate image), the components O and P refer to the same image data. A liaison or link between the image data and its component salient objects can be maintained with a reference by a key or *id*. A can in addition be used to store some spatial, topological and hierarchical relationships among the salient objects and that of the image. Similarity-based operations on salient-objects is then performed with reference to each of the F_s components. The similarity-based algebraic operators we discussed in this paper can be used to support operations on salient objects.

8 Query Expressions Involving both Similarity-Based and Relational Operations

Using our image data repository model and considering the similarity-based operations we introduced, we state below query statements that demonstrate the way we can manage the queries stated in Section 2. We want to note that the result of each query is an image table. To display the images them selves, a visual interface built on the top of the query system is required. Since our operators and our similarity-based algebra are novel introductions to an Image DBMS, none of the existing query languages support all our operators. Multimedia Object Query Language (MOQL) introduced in [15] uses an operator called "*Contains*" for a similarity-based image re-

trieval. However, it is not an operator that can perform our requirements for similarity-based join.

As discussed in the previous sections, our similarity-based operators use the parameter ε for the range query. We suppose that this value is defined in the query system before stating the query expression and is not mentioned in the query statement. As discussed in Section 6, similarity-based joins maintain the referring instances of an image table and capture the referenced or associated instances in the other table using the pointer component P. Note that images can only be displayed using visual interfaces. Visualizing results of a content-based query is at a higher level to the algebraic expressions. We present here only the query expressions without considering visualizing the images in the resulting table.

Consider the example queries in Section 2. For queries No. 2 and 3., we formulate the image tables as follows:

$SI(id, Surv_Photo, F, A', P)$, where A' is an object component given by A' ={Date, Time}[4] $Surv_Photo$ is the photo taken by the surveillance camera.

$EMP(id, Photo, F, A^*, P)$, where the A^* component is composed of a schema A^*(ename, eaddress, Dept, Occupation). $Photo$ is the photo of the employee. Then, the query statement for query 2 can be stated using an SQL like query language as:

```
SELECT     *
      FROM SI, EMP
      WHERE SI.F ≃[5] EMP.F
         AND  EMP.Dept = 'Production'.
```

For Query 3, we can state the statement as:

```
SELECT     *
      FROM SI, EMP
      WHERE   SI.F ≃ EMP.F
         AND   SI.A'.Date = 11-09-2001
         AND   SI.A'.time  BETWEEN  (6:00 AND 10:00AM).
```

The image tables can be re-formulated for Query 4 of Section 2. as:
SIE(id, O, F', A^e, P),
SII(id, O, F*, A*, P), where A^e and A* contain the date and time and the surveillance images are taken at the exterior and interior gates respectively. The EMP table given above can be considered as it is for Query 4 too.

Then, the query expression for Query 4 of Section 2. is:

```
SELECT    *
FROM   SIE, SII, EMP
WHERE  SIE.F ≃ SII.F
          AND  SIE.F ≃ EMP.F
          AND  SI.A'.Date = 11-09-2001
          AND  SI.A'.time  BETWEEN  (6:00 AND 10:00AM).
```

[4]Some database systems store both Date and time under the same data structure. We however state it here separately for the purpose of clarity.

[5] *The symbol ≃ is associated to the similarity-based join operation $\oplus^{\varepsilon}$ discussed in the previous sections.*

Let us now consider a case in Bio-Medical Application. Suppose a specialist is interested to analyze medical images collected from different sources in comparison with and existing medical image database of Thesaurus. Due to the limitations of describing an image using key words, supporting medical applications with content-based image description and retrieval techniques is of high importance. The similarity-based operations we proposed in this paper can then be applied in such kinds of Bio-medical image analysis and classification applications.
Consider that the collected images are lung x-ray images of patients and it is organized in an image table P. The table P of patients doesn't contain adequate information on treatments and no interpretations are given for the anomalies. Let the thesaurus table T contain a large collection of medical images with identified anomalies, the corresponding diagnosis and the treatments given.

We here see that the only means to get information on identifying the anomalies, possible diagnosis and identifying the experience on treating such cases is by making a similarity-based join on the representations of the images or salient objects in the two tables, P and T. Anomalies are parts of images and hence are identified as salient objects. We assume here that the image tables are structured based on the model we presented in this paper. Hence, the S_P and S_T tables given below are tables that contain all the information related to the identified salient objects in P and T respectively. Suppose that P is created for the purpose of this analysis and its F component is automatically extracted and its salient objects are identified and stocked in S_P. The image tables involved in this problem are given as:

$P(id, O, F, A^p, P)$,
$S_P(id_s, F_s, A_s)$, where S_P is a table that contains the salient objects of the images in P.
$T(id, O, F, A^t, P)$,
$S_T(id_s, F_s, A_s)$, where S_T is a table that contains the salient objects of the images in T.

The A components of the above tables can be stated as:

$A^p(Name, Gender, Age, Address, Image_Type, S_id)$,
$A^t(Image_Type, Date, Physician, Treatment, S_id)$, where the S_id's contain all corresponding id's of S_P and S_T whose salient objects belong to the image of P.id.

The A component of S_T is given as:

$A_s(anomaly_type, Anomaly_description, S_T_id)$

The major difference of P and T is that in T and S_T anomalies are well identified and the complete information on the diagnosis and treatment of the case is well documented. The specialist is therefore interested to exploit the expertise used in T in identifying and classifying the similar cases of the images in P. Then, the specialist may for example have a query stated as:

"Retrieve all personal data of patients and the treatments given to its similar on lung related medical cases as shown by the X-ray images in the patient and thesaurus tables".

The expression for this query can be given as follows:

SELECT *
FROM P, T, S_P, S_T
WHERE $(P.A^p.Image_Type = "X-ray")$
AND $(T.A^t.Image_Type = "X-ray")$
AND $(S_P.F_s \simeq S_T.F_s)$

It is evident that such queries need to be supported with visual query interfaces that can manage the display of the images whenever required. Developing such an interface is not our current task. Such interfaces are developed in many systems [9]. Our task here is to develop the fundamental modeling requirements and algebraic formalisms for a system that uses both content-based and relational query in an integrated manner in image databases.

9 Similarity-Based Query Optimization

A query in an image database system is quite different from a query in the relational database systems. Besides the fact that, queries can contain image object(s) as an input given by the user, the results of image queries are not based on perfect matches but on degrees of similarity. Moreover, considering the fact that most feature extraction and content-based retrieval algorithms are very expensive, the need for query optimization of image databases is very critical.

9.1 Algebraic Rules on the Similarity-Based Operations

To develop query optimization strategies for content-based image database systems, a well defined algebra for the similarity-based operations is necessary. Our work to introduce the necessary novel similarity-based operators and to study their properties for a possible similarity-based query operation in [2] and in this paper targets to formalize the similarity-based algebraic space. Thus, the properties of these similarity-based algebraic operators with the properties of the already existing relevant relational operators can be utilized to formulate query optimization rules for content-based image databases. Such algebraic rules plays an important role to query optimization in addition to indexing schemes. Below are the major rules that can be applied for query optimization purpose.

9.1.1 The Properties of Pushing Selection

We separately consider two types of selection operators. A relational selection, for example on the "A" component, and a similarity-based selection on the "F" component of an image table. Let us denote the relational selection by $\sigma_{M.a}$ and the similarity-based selection by $\sigma_{M.f}$, where "a" is an attribute value and "f" is a feature vector.

The following properties hold true:

1. $\sigma_{M_1.f}(M_1 \uplus M_2) = \sigma_{M_1.f}(M_1) \uplus M_2$; pushing a similarity-based selection in to an additive union,
2. $\sigma_{M_1.f}(M_1 \otimes^{\varepsilon} M_2) = \sigma_{M_1.f}(M_1) \otimes^{\varepsilon} M_2$; pushing a similarity-based selection in to a similarity-based join,
3. $\sigma_{M_1.f}(M_1 \oplus^{\varepsilon} M_2) = \sigma_{M_1.f}(M_1) \oplus^{\varepsilon} M_2$; pushing a similarity-based selection in to a symmetric similarity-based join (this follows from the above two).

These properties hold true also for the relational selection operator, $\sigma_{M.a}$.

9.1.2 Transformation Rules on Similarity-Based Join

The properties of the similarity-based binary operators defined and discussed in Section 4 can be used to formulate the following rules for query optimization.

1. The commutative property of $\oplus$.
   Whenever, the symmetric join $\oplus$ is applied, its commutative property can be used to optimize query processing.
2. Reordering operand tables in a symmetric multi similarity-based join.
   i.e. the order of the operand tables can be rearranged for an optimal query processing.
   The use of the $Mine$ operator can as well be considered in the above two rules.
3. The transformation property to exchange order using $Mine$.
   (i.e. $M_1 \otimes M_2 \rightarrow Mine(M_2 \otimes M_1)$).
   Even in the non-symmetric similarity-based operator, we can use the $Mine$ operator for an optimized query processing.
4. Join method choice :
   $M_1 \otimes_{method_1} M_2 \rightarrow M_1 \otimes_{method_2} M_2$;
   An optimum join algorithm that results to the same output table can be chosen.

The factors for the exchange of the order of the operand tables could be: the number of instances in the tables, the density of the objects in the feature space, the value of ε, the amount of I/O and Memory requirement, etc. A query optimizer must therefore carefully compare the benefits of transforming an expression against the computing costs.

9.2 Local Optimization

We refer a local optimization as the relative inner table choice in a non-symmetric similarity-based join. For instance, it could be less costly to compute $M_2 \otimes^{\varepsilon} M_1$ than $M_1 \otimes^{\varepsilon} M_2$ or vice-versa. Based on Rule 3 above, the supplemental price for the exchange of order is the cost of the $Mine$ operator. That is, instead of computing $M_1 \otimes^{\varepsilon} M_2$, we compute first the $M_2 \otimes^{\varepsilon} M_1$ and then apply the $Mine$ operator on the result. It is the task of the query optimizer to decide which one-sided similarity-based join to perform first and then use the $Mine$ operator for the other. The decision depends on the cost model for the similarity-based join. A cost model for a simple nested-loop similarity-based join implementation is presented in the next subsection. This cost model is built based on the previously developed cost models [5, 14] for content-based image retrieval. Based on this model:
$cost(M_1 \otimes^{\varepsilon} M_2) = \frac{N_1}{C_{eff}}(1 + N_2 \cdot \sum_{j=1}^{d'} \binom{d'}{j} \cdot (\frac{1}{2})^j \cdot S_{\varepsilon}^{d'-j})$,

with N_1, N_2 being the number of data points in M_1 and M_2 respectively, C_{eff} being the effective capacity of a leaf partition and d' be computed as $d' = \lceil log_2(N_2/C_{eff}) \rceil$. A uniform object distribution is considered in this model..

Obviously, the cost of similarity-based join depends linearly on the number of data points in M_1. However, the dependency in the number of data points in M_2 is far less obvious. This means, that compared to relational query optimization, no simple decision rule can be established which decides on the relative inner table choice to one similarity-based join. Therefore, we require to compute the cost of each alternative, separately and then decide which strategy to follow.

Consider for example the transformation rule: $M_1 \otimes^{\varepsilon} M_2 \rightarrow Mine(M_2 \otimes^{\varepsilon} M_1))$. This transformation rule is worth-full only if cost$(M_1 \otimes^{\varepsilon} M_2) > cost(Mine(M_2 \otimes^{\varepsilon} M_1))$. Compared to the respective cost of computing the non-symmetric join operation, the cost of $Mine$ could be negligible. Therefore, a test of the form: **if cost**$(M_1 \otimes^{\varepsilon} M_2) >$ **cost**$(M_2 \otimes^{\varepsilon} M_1)$ **then perform** $Mine(M_2 \otimes^{\varepsilon} M_1)$; can mostly be applied.

9.3 Cost Model

The common content-based image retrieval operations to date is of type: given an image, find its all similar images from a set of or a database of images. The complexity of content-based image representations has initiated the use of multidimensional indexing schemes [6]. A multidimensional index structure makes it possible to access only fewer index pages in order to increase the efficiency of content-based image retrieval. Existing index structures for high dimensional feature spaces can be classified into Data Partitioning (DP)-based and Space Partitioning (SP)-based structures [8].

Based on a simple and straightforward nested-loop implementation of similarity-based join using the range-search, we develop a cost model that works with DP-based index structures and that uses a hierarchical index structure. The straightforward method for performing a similarity-based join is to directly apply the algorithm of a range query for each object of the left input table M_1 as a query object looking for its similar objects from the right input table M_2.

The metric used is the number of required disk accesses for performing the similarity-based join. This is a common metric in use in related works [5, 14].

We assume that the objects of the two operand image tables M_1 and M_2 follow the same object distribution function over a normalized object space of $[0, 1)^d$ (d is the dimension of the feature vector) and that a DP-based index structure is used. The impact of a concrete distribution function is discussed after the general cost considerations. The notations used here are shown in the table below.

Symbols	Notations
M_1, M_2	operand image tables
d	Dimension of the feature vectors in M_1 and M_2
N_1, N_2	Number of data points in M_1 and M_2 respectively
m_1, m_2	Number of leaf nodes in the index structure of M_1 and M_2 respectively
o_1^k	k-th object in the index structure of M_1
x_2^i	length vector of the i-th leaf node Li of M_2, i.e. $x_2^i = (x_2^{i\,1}, \ldots, x_2^{i\,d})(1 \leq k \leq d)$
C_{eff}	average number of data points in one leaf page of M_2
V_{ε}^j	hyper-volume of the hypersphere S_{ε}^j, with radius ε for a dimension $0 \leq j \leq d$
$DA(o_1^k)$	expected disk accesses for the range-search of an object o_1^k in M_2
DA_{total}	expected total number of disk accesses

The expected number of disk accesses for a similarity-based join is the sum of expected disk accesses for the range search of all objects o_1^k of M_1 $(1 \leq k \leq N_1)$ in M_2. The sum of disk accesses of one object o_1^k of M_1 is the number of leaf partition nodes of M_2 that intersects the hypersphere S_{ε}^d containing the neighbors for o_1^k within the distance of ε.

In order to compute the number of leaf partition nodes of M_2 that intersects S_{ε}^d the Minkowski Sum of S_{ε}^d and the leaf nodes is determined. The Minkowski Sum corresponds graphically to the

volume of an area which results from moving the center of S_ε^d over the surface of the bounding box of one leaf node. Summing up the Minkowski Sum for each leaf node results in the expected number of disk accesses.

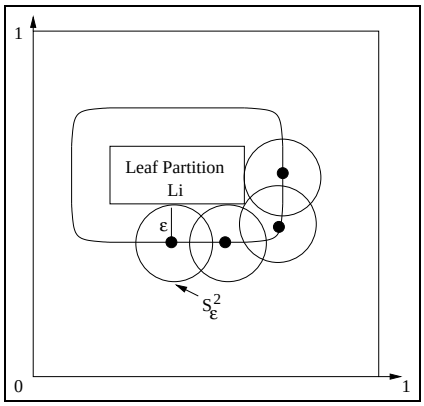


Figure 6: Minkowski Sum in a 2-dimensional space.

For instance, consider figure 6 for the Minkowski Sum in a 2-dimensional space. One leaf node Li of M_2 is a rectangle with length vector $x_2^i = (x_2^{i\,1}, x_2^{i\,2})$. The query circle S_ε^2 with radius ε is the region which includes the neighbors of the query object. The number of disk accesses for the query object is the sum of the probabilities for intersection with all leaf partitions. The probability that S_ε^2 intersects one leaf partition Li corresponds graphically (see figure 6) to an area which results from moving the circle S_ε^2 around the leaf partition Li. The volume of this area, i.e. the probability that S_ε^2 intersects Li, is: $volume(Li) + perimeter(Li) \cdot \varepsilon + V_\varepsilon^2$.

Summing it up to the index structure with m_2 index nodes, we obtain the expected number of disk accesses $DA(o_1^k)$ for the range-search of an object o_1^k which expresses in the 2-dimensional case as :

$$\sum_{i=1}^{m_2} volume(Li) + perimeter(Li) \cdot \varepsilon + volume(S_\varepsilon^2) = \sum_{i=1}^{m_2} x_2^{i\,1} \cdot x_2^{i\,2} + x_2^{i\,1} \cdot \varepsilon + x_2^{i\,2} \cdot \varepsilon + \pi \cdot \varepsilon^2.$$

Expressing the last line in terms of hyperspheres S_ε^j of dimensions lower than d, we obtain a slightly modified formulae that corresponds exactly to the Minkowski Sum of S_ε^d : $\sum_{i=1}^{m_2} x_2^{i\,1} \cdot x_2^{i\,2} \cdot S_\varepsilon^0 + x_2^{i\,1} \cdot \cdot S_\varepsilon^1 + x_2^{i\,2} \cdot S_\varepsilon^1 + \pi \cdot S_\varepsilon^2$

In the general case, $DA(o_1^i)$ computes than as :

(1) $DA(o_1^k) = \sum_{i=1}^{m_2} \sum_{j=1}^{d} \sum_{\{y^1,\ldots,y^j\} \in PowerSet\{x_2^{i\,1},\ldots,x_2^{i\,d}\}} \cdot volume(\{y^1,\ldots,y^j\}) \cdot S_\varepsilon^{d-j}$

where,

(a) $S_\varepsilon^j = \frac{\sqrt{\pi^n}}{\Gamma(n/2+1)} \cdot \varepsilon^j$,

(b) $volume(\{y^1,\ldots,y^j\}) = \prod_{k=1}^{j} y^k$

In order to obtain a cost model for high-dimensional spaces, we have to consider the so-called boundary effects (i.e. the perimeter of the bounding box of one leaf node of M_2 is partially outside the data space $[0,1)^d$ and does consequently not contribute to the probability that this leaf node intersects the query object [5]). The main idea to deal with this problem is to attribute significance to the dimensions. i.e. a dimension is significant if its part in the Minkowski sum contributes to the overall probability. It has been shown by Berchtold et al. in [5] that only the first d' dimension are significant with $d' = \lceil log_2(N_2/C_{eff}) \rceil$. Consequently, equation (1) may be rewritten as follows in order to suit to boundary effects.

(2) $DA(o_1^k) = \sum_{i=1}^{m_2} \sum_{j=1}^{d'} \sum_{\{y^1,\ldots,y^j\} \in PowerSet\{x_2^{i\,1},\ldots,x_2^{i\,d'}\}} \cdot volume(\{y^1,\ldots,y^j\}) \cdot S_\varepsilon^{d'-j}$

We can now express the expected total disk accesses, DA_{total}. For each object o_1^i of M_1 we compute the neighbors within distance ε in M_2. We can obviously assume that for two objects located in the same leaf node of M_1, the leaf node is still in the cache. Therefore DA_{total} computes as :

(3) $$DA_{total} = m_1 + \sum_{k=1}^{N_1} DA(o_1^k)$$

It is a common use in relational databases to estimate uniform object distribution, for instance, for attribute values in order to compute the join selectivity. We will adopt this principle here. However, we are aware that this gives only a rough estimation, as many applications do not follow an uniform distribution [5, 14]. Uniform object distribution means that the object points follow a random distribution in the normalized object space of $[0,1)^d$.

The number of leaf nodes in the index structure of M_2, m_2, is computed by fixing an index page size of B Bytes (typically 4096 Bytes). This implies an effective capacity of $C_{eff} = \frac{B}{d \cdot V}$ data objects per data page, supposing that the value in each dimension is stored in V Bytes. Consequently, $m_2 = \frac{d \cdot V \cdot N_2}{B}$ and $d' = \lceil log_2(\frac{N_2 \cdot d \cdot V}{B}) \rceil$. Finally, the length vector x_2^i of all leaf node in M_2 is $(1/2, \ldots, 1/2)$, i.e. $\forall_{k(1 \leq k \leq d)}, x_2^{i\,k} = 1/2$. Respectively, m_1 is computed as $m_1 = \frac{d \cdot V \cdot N_1}{B}$, supposing the same dimension feature vectors, as well as its representation in the same index structure. Equation (3) can then be reduced to a simplified version (using N_1 and N_2) for a uniform object distribution as :

(4) $$DA_{total} = \frac{N_1}{C_{eff}}(1 + N_2 \cdot \sum_{j=1}^{d'} \binom{d'}{j} \cdot (\frac{1}{2})^j \cdot S_\varepsilon^{d'-j}).$$

10 Conclusions

The many successful research results in the domain of image analysis and pattern recognition have made content-based image retrieval a promising approach. The integration of these facilities into DBMSs in order to support content-based image data management is what should follow next. One of the key issues to tackle this problem is the existence of a well-formalized similarity-based algebra usable for modeling, optimizing and processing *similarity-based queries*. In this view, we first have summarized the works done by different research and commercial groups on content-based image retrieval. Then, we presented a new schematic model for an image data repository under an OR paradigm. In this model, a new pointer component called P is introduced to capture links between image tables during binary operations. An extension of this model to support operations on salient image objects is also proposed. We have defined several novel basic similarity-based and supportive operators and studied their properties. The possible techniques of using these similarity-based operators in combination with relational operators are also investigated. Finally, we investigated query optimization possibilities based on our novel system of similarity-based operators. Novel algebraic rules useful for query optimization are identified and a cost model for a simple similarity-based join algorithm is derived. This work lays down the foundations for further work on an integrated content-based and relational image data management system.

Future work include, the development of an application prototype based on our similarity-based operators, the design of a visual query interface and the design of a similarity-based query optimizer model.

Acknowledgment:
We would like to thank Prof. Jacques Kouloumdjian of LISI, INSA de Lyon, for his valuable comments and encouragements.

References

[1] S. Adali, P. Bonatti, M. L. Sapino, and A. S. Subrahmanian. A Multi-Similarity Algebra. *SIGMOD Conference*, pages 402–413, 1998. Seattle, Washington, USA.

[2] S. Atnafu, L. Brunie, and H. Kosch. Similarity-Based Operators in Image Database Systems. In *WAIM'2001*, pages 14–25, Xi'an, China,, July 2001. LNCS 2118.

[3] S. Berchtold, C. Boehm, B. Braunmueller, D. A. Keim, and H. P. Kriegel. A Cost Model for Nearest Neighbor Search in High-Dimensional Data Space. *ACM PODS*, pages 78–86, 1997. Arizona.

[4] S. Berchtold, C. Boehm, B. Braunmueller, D. A. Keim, and H. P. Kriegel. Fast Parallel Similarity Search in Multimedia Databases. *SIGMOD Conf.*, pages 1–12, 1997. AZ,USA.

[5] S. Berchtold, C. Bohm, D. A. Keim, and H.P. Kriegel. A cost model for nearest neighbor search in high-dimensional data space. In *Proceedings of the ACM PODS Conference*, pages 78 – 86, Tucson, Arizona, May 1997.

[6] S. Berchtold, Daniel A. Keim, and Hans-Peter Kriegel. The X-tree: An indexing structure for high-dimensional data. In *Proceedings of the VLDB Conference*, pages 28–39, Bombay, India, September 1996.

[7] Alberto Del Bimbo. *Visual Information Retrieval.* Morgan Kaufmann Publishers Inc., San Fransisco, California, 1999. ISBN 1-55860-624-6.

[8] K. Chakrabarti and S. Mehrotra. The Hybrid Tree: An index structure for high dimensional feature spaces. In *Proceedings of the Intl. Conference on Data Engineering (ICDE)*, pages 440–447, Sydney, Australia, March 1999.

[9] John P. Eakins and Margaret E. Graham. Content-Based Image Retrieval: A report to the JISC Technology Applications Programme, January 1999. Inst. for Image Data Research, Univ. of Northumbria at Newcastle.

[10] G.R. Hjaltason and H. Samet. Ranking in spatial databases. *Lecture Notes in Computer Science 951*, pages 83–95, 1995. Springer-Verlag, Berlin.

[11] J.K.Wu, A.D. Narasimhalu, B.M. Mehtre, C.P. Lam, and Y.J. Gao. CORE: A Content-Based Retrieval Engine for Multimedia Information Systems. *Multimedia Systems*, 3:25–41, 1995.

[12] H. Kosch, L. Brunie, and W. Wohner. From the Modeling of Parallel Relational Query Processing to Query Optimization and Simulation. *Parallel Processing Letters*, 8(1):2–14, 1998.

[13] Nick Koudas and Kenneth C. Sevcik. High Dimensional Similarity Joins: Algorithms and Performance Evaluation. *IEEE Transactions on Knowledge and Data Engineering*, 12(1):3–18, 2000.

[14] J.-H. Lee, G.-H.Cha, and C.-W. Chung. A model for k-Nearest Neighbor query processing cost in multidimensional data space. *Information Processing Letters*, 69(2), 1999.

[15] V. Oria, M.T. zsu, P. Iglinski, S. Lin, and B. Yao. DISMA: A Distributed and Interoperable Image Database System. In *SIGMOD 2000*, Dallas, Texas, May 2000. In Proc. of ACM SIGMOD Int. Conf. on Management of Data.

[16] Informix Press. Excalibur Image Datablade Module User's Guide, March 1999. Ver. 1.2, P. No. 000-5356.

[17] Oracle Press. Oracle8i Visual Information Retrieval Users Guide & Reference, 1999. Release 8.1.5, A67293-01.

[18] Mirela Tanase Remco C.Veltkamp. Content-Based Image Retrieval Systems: A Survay, October 2000. Technical Report UU-cs-2000-34, Department of Computer Science, Utrecht University.

[19] Y. Rui, T.S. Huang, and S.F. Chang. Image Retrieval: Past, Present, and Future. *Journal of Visual Communication and Image Representation*, 10:1–23, 1999.

[20] T. Seidl and H.P. Kriegel. Efficient user-adaptable similarity search in large multimedia databases. *VLDB'97, Athens, Greece*, pages 506–515, 1997.

[21] T. Seidl and H.P. Kriegel. Optimal Multi-Step k-Nearest Neighbor Search. *SIGMOD*, pages 154–165, 1998. WA, USA.

[22] Amit Sheth and Wolfgang Klas. *Multimedia Data Management: Using Metadata to Integrate and Apply Digital Media.* McGraw-Hill, San Francisco, 1998.

[23] M. Stonebraker and P. Brown. *Object-Relational DBMSs.* Mogan Kaufmann Pub. Inc, San. Francisco, 1999. ISBN 1-55860-452-9.

[24] V. S. Subrahmanian. *Principles of Multimedia Database Systems.* Morgan Kaufmann Publishers Inc., San Fransisco, California, 1998. ISBN 1-55860-466-9.

[25] V.Oria, M.T. zsu, L. Liu, X. Li, J.Z. Li, Y. Niu, and P.J. Iglinski. Modeling Images for Content-Based Queries: The DISMA Approach. *VIS'97, San Diago*, pages 339–346, 1997.

[26] Jian-Kang Wu. Content-Based Indexing of Multimedia Databases. *IEEE TKDE*, 9(6):978–989, 1997.

[27] A. Yoshitaka and T. Ichikawa. A Survey on Content-Based Retrieval for Multimedia Databases. *IEEE Transactions on Knowledge and Data Engineering*, 11(1):81–93, 1999.